BÖLÜM 4 / MODULE 4

Ağ, Keşif & Dış Dünyaya Açılma

Networking, Discovery & Ingress Routing

Pod'lar arası dinamik IP kaosuna son: ClusterIP servisleri, NodePort, LoadBalancer, Ingress kontrolcüleri, CoreDNS ve ConfigMap/Secret yönetimi.

Taming ephemeral IP chaos: ClusterIP services, NodePort, LoadBalancers, Ingress traffic cops, CoreDNS resolution, and externalizing configs via ConfigMaps & Secrets.

19

BÖLÜM: AĞ, KEŞIF & DIŞ DÜNYAYA AÇILMA | MODULE: NETWORKING, DISCOVERY & INGRESS ROUTING

Uçucu IP Çilesine Son: Service (ClusterIP) ve Pod Keşfi

Solving Ephemeral IPs: Services (ClusterIP) & Internal Discovery

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Pod'lar kelebekler gibidir; ömürleri kısadır, sürekli ölür ve her yeniden doğduklarında farklı bir telefon numarası (IP adresi) alırlar. Müşterilerinize sürekli değişen cep numaraları veremezsiniz! 'Service' (ClusterIP), şirketin asla değişmeyen sabit santral numarasıdır (Örn: 444 0 444). Müşteri santrali arar, santral arkada o an hangi garson müsaitse telefonu ona aktarır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Pods are butterflies: ephemeral and fragile. Every time a pod dies and resurrects, it receives a completely new IP address. You can't distribute changing phone numbers to customers! A Service (ClusterIP) is your company's immutable 1-800 hotline. Callers dial the single hotline number, and the switchboard automatically forwards calls to whoever is on duty!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

ClusterIP, sadece küme içinden erişilebilen sanal bir IP (VIP) ve DNS adıdır. Fiziksel bir ağ arayüzü yoktur! Arka planda `kube-proxy`, Linux çekirdeğindeki iptables veya IPVS kurallarını güncelleyerek ClusterIP'ye gelen paketleri hedef pod'ların IP'lerine rastgele (Round-Robin) yönlendirir. Servis, pod'ları `selector.matchLabels` ile bulur ve bu pod'ların canlı IP'lerini 'EndpointSlice' nesnelerinde güncel tutar.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

A ClusterIP is a virtual IP (VIP) resolvable strictly within the cluster. It has no physical network card! Instead, `kube-proxy` configures iptables or IPVS packet filter rules in the Linux kernel to rewrite destination IPs to matching pod IPs via round-robin. The Service maintains an active inventory of backing Pod IPs through `EndpointSlice` controller watchers.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# coffee-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: coffee-service # Küme içi DNS adı bu olur!
spec:
  type: ClusterIP
  selector:
    app: coffee-app # Bu etikete sahip pod'lara yönlendir
  ports:
    - name: http
      port: 80         # Servisin dinlediği dış kapı
      targetPort: 8080 # Spring Boot konteynerinin portu
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

ClusterIP servisine küme dışından (laptopunuzun tarayıcısından) doğrudan erişemezsiniz! Sadece küme içindeki diğer pod'lar veya `kubectl port-forward` ile erişilebilir.

🇬🇧 WATCH OUT

ClusterIP addresses are strictly internal! You cannot curl or open them in your desktop browser without port-forwarding or external ingress.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Service, dinamik pod IP'lerinin önüne sabit bir sanal IP ve DNS koyar.
- Trafik arka planda sağlıklı pod'lar arasında otomatik dengelenir.
- EndpointSlice nesneleri hazır olan pod'ların IP listesini anlık tutar.

- Services establish an immutable virtual IP and DNS fronting dynamic pods.
- Traffic load balances evenly across healthy pods matching label selectors.
- EndpointSlices continuously track ready and healthy pod endpoints.

20

BÖLÜM: AĞ, KEŞIF & DIŞ DÜNYAYA AÇILMA | MODULE: NETWORKING, DISCOVERY & INGRESS ROUTING

Dış Dünyaya İlk Kapı: NodePort ve LoadBalancer

Exposing Apps Outside: NodePort and Cloud LoadBalancers

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

ClusterIP ile evin içindesiniz. Peki sokaktaki müşteri içeri nasıl girecek? 1) NodePort: Binanın dış duvarına rastgele yüksek bir delik açıp kablo sarkıtmaktır (Port 30000-32767). 2) LoadBalancer: Binanın önüne profesyonel bir vale ve döner kapı koymaktır (AWS, Google Cloud veya Azure'dan gerçek bir genel IP kiralar)!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

ClusterIP keeps you locked indoors. How does a customer on the street enter? 1) NodePort: Drilling a hole in the outside wall with a designated high-numbered port (30000-32767). 2) LoadBalancer: Hiring an elite cloud-native valet company to manage your driveway with a shiny public static IP (AWS NLB, GCP Load Balancer)!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

1) `NodePort`: Kümedeki TÜM işçi düğümlerin belirtilen portunu (örn: 31234) dinlemeye alır. `http://<HERHANGİ_BİR_DÜĞÜM_IP>:31234` adresine gelen istek servise aktarılır. 2) `LoadBalancer`: NodePort'un bir üst katmanıdır. K8s bulut sağlayıcısının (AWS/GCP/Azure) API'sini çağırarak otomatik bir bulut yük dengeleyici tahsis eder ve trafiği küme düğümlerine dağıtır. Yerel ortamlarda (Minikube/Kind) MetalLB kullanılabilir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

1) `NodePort` opens a dedicated static port (30000-32767) on EVERY worker node. Packets hitting `<NodeIP>:<NodePort>` route into the service. 2) `LoadBalancer` extends NodePort by dispatching cloud provider API drivers (AWS ELB, GCP CLB) to provision a public IP routing external Internet traffic into the node ports.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# coffee-loadbalancer.yaml
apiVersion: v1
kind: Service
metadata:
  name: coffee-public-service
spec:
  type: LoadBalancer # Buluttan genel IP talep et!
  selector:
    app: coffee-app
  ports:
    - port: 80
      targetPort: 8080

# Minikube'da test etmek için tünel açma:
# minikube tunnel
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Her mikroservis için ayrı bir `LoadBalancer` servisi açarsanız, bulut sağlayıcınız her biri için ayda onlarca dolar fatura keser! 20 mikroservis için 20 ayrı IP almak yerine tek bir `Ingress` kullanılmalıdır.

🇬🇧 WATCH OUT

Provisioning a cloud LoadBalancer for every single microservice explodes cloud bills! Use a single Ingress Controller to route 50 services through 1 public IP.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- NodePort geliştirme ve test ortamlarında hızlı dış erişim sağlar.
- LoadBalancer gerçek dünya üretim ortamlarında genel IP sağlar.
- Trafik önce LoadBalancer'a, oradan NodePort'a, oradan ClusterIP'ye akar.

- NodePort is useful for quick debugging without cloud provider dependencies.
- LoadBalancers automate cloud-native public IP allocation and routing.
- Traffic hierarchically flows: LoadBalancer -> NodePort -> ClusterIP -> Pod.

21

BÖLÜM: AĞ, KEŞIF & DIŞ DÜNYAYA AÇILMA | MODULE: NETWORKING, DISCOVERY & INGRESS ROUTING

Ingress & Ingress Controller: Akıllı Trafik Polisi

Ingress & Ingress Controllers: The Smart Traffic Cop

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

50 katlı bir holding binasında her ofis için ayrı sokak kapısı yapamazsınız! Binanın tek bir ana döner kapısı ve lobisinde akıllı bir danışma görevlisi (Ingress Controller) olur. Gelen kişiye sorar: 'Kime geldiniz?' Kişi 'Kafeye (`/coffee`)' derse kafeye, 'Muhasebeye (`/billing`)' derse 5. kata yönlendirir. Üstelik kapıda kimlik kontrolü (SSL/TLS şifreleme) yapar!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

You don't build 50 separate front doors for a 50-story skyscraper! You have 1 grand lobby entrance with a sharp receptionist (Ingress Controller). Guests state their destination: 'Going to the coffee shop (`/coffee`)?' Turn left. 'Accounting (`/billing`)?' Elevator 4. Plus, the receptionist checks IDs and encrypts passes (HTTPS / SSL termination)!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Ingress iki bileşenden oluşur: 1) Ingress Kaynağı (Kuralları belirten YAML dosyası), 2) Ingress Controller (Bu kuralları dinleyip uygulayan NGINX, Traefik, Envoy veya HAProxy pod'u). Ingress Controller, 7. katman (HTTP/HTTPS) yönlendirmesi yapar: Domain adına göre (`api.sirket.com` vs `web.sirket.com`) veya URL yoluna göre (`/api` vs `/auth`) trafiği ilgili ClusterIP servisine aktarır. SSL sertifikası tek merkezde sonlandırılır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Ingress splits into: 1) Ingress Resource (declarative routing rules in YAML), 2) Ingress Controller (reverse proxy daemon like Ingress-NGINX or Traefik reconciling those rules). Operating at Layer 7 (HTTP/HTTPS), it terminates TLS, rewrites paths, and directs traffic based on SNI hostnames and URL path prefixes to backend ClusterIPs.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# coffee-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: coffee-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
    - host: coffee.local
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: coffee-service
                port:
                  number: 80
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Ingress YAML'ı yazmak tek başına HİÇBİR İŞE YARAMAZ! Kümenizde çalışan aktif bir Ingress Controller (örn: ingress-nginx) yoksa, yazdığınız Ingress kuralları sessizce havada asılı kalır.

🇬🇧 WATCH OUT

Applying an Ingress manifest without an active Ingress Controller running in your cluster does absolutely nothing; the resource remains dormant.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Ingress tek bir genel IP arkasında yüzlerce servisi akıllıca paylaştırır.
- Host-tabanlı ve Path-tabanlı 7. katman yönlendirmelerini destekler.
- SSL/TLS sertifikalarını tek merkezde yönetip güvenliği artırır.

- A single Ingress exposes hundreds of microservices behind 1 public IP.
- Supports host-based (virtual hosting) and path-based Layer 7 routing.
- Centralizes TLS termination and SSL certificate lifecycle management.

22

BÖLÜM: AĞ, KEŞIF & DIŞ DÜNYAYA AÇILMA | MODULE: NETWORKING, DISCOVERY & INGRESS ROUTING

Servisler Arası İletişim & CoreDNS (Spring Boot Entegrasyonu)

Inter-Service Communication & CoreDNS Resolution

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

K8s içindeki akıllı telefon rehberi: CoreDNS! Bir Spring Boot mikroservisi diğerine istek atarken IP adresini bilmek zorunda değildir. Rehbere 'Bana sipariş servisini bağla!' (`order-service`) der, CoreDNS anında doğru numarayı bağlar. Hatta başka bir departmandaysa tam adını söyler: `coffee-service.coffee-shop.svc.cluster.local`!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The internal yellow pages: CoreDNS! When your Spring Boot frontend calls your backend, it doesn't need to track IP addresses. It simply calls `http://coffee-service`! CoreDNS intercepts the DNS request and resolves the cluster IP instantly. Across namespaces, it uses the Fully Qualified Domain Name (FQDN): `service.namespace.svc.cluster.local`!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

K8s kümesindeki her Pod'un `/etc/resolv.conf` dosyasına CoreDNS servisinin IP'si otomatik yazılır. Bir pod `http://coffee-service` çağırdığında: 1) Yerel libc, search domain listesini (`coffee-shop.svc.cluster.local`, `svc.cluster.local`, `cluster.local`) sırayla dener. 2) CoreDNS eşleşen Servis VIP'sini döner. 3) Kube-proxy paketi hedef pod'a iletir. Spring Boot'ta `RestTemplate` veya `WebClient` ile doğrudan servis adı çağrılır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Every Pod automatically inherits `/etc/resolv.conf` pointed at the cluster's CoreDNS service IP. Calling `http://coffee-service` triggers iterative DNS search domain appending (`<ns>.svc.cluster.local`). CoreDNS returns the Service VIP, and kube-proxy handles Layer 4 routing. In Spring Boot, configure your `WebClient` base URL directly with the service DNS name.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
// Spring Boot Java Servis Çağrısı Örneği:
@Service
public class OrderService {
    private final RestClient restClient;

    public OrderService(RestClient.Builder builder) {
        // CoreDNS adı doğrudan URL olarak kullanılır!
        this.restClient = builder
            .baseUrl("http://coffee-service.coffee-shop.svc.cluster.local")
            .build();
    }

    public CoffeeResponse fetchCoffee() {
        return restClient.get()
            .uri("/api/coffee")
            .retrieve()
            .body(CoffeeResponse.class);
    }
}
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Aynı namespace içindeki servisleri sadece adıyla (`http://coffee-service`) çağırabilirsiniz. Ama FARKLI bir namespace'teki servisi çağırmak için mutlaka namespace adını da eklemelisiniz (`http://coffee-service.diger-namespace`)!

🇬🇧 WATCH OUT

Calling a service in another namespace requires appending the namespace: `http://service-name.target-namespace`. Omitting it causes DNS 404 lookup failures.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- CoreDNS tüm servisler için otomatik DNS A/SRV kayıtları oluşturur.
- Tam alan adı: `<service-name>.<namespace>.svc.cluster.local` şeklindedir.
- Spring Boot kodunda IP adresi yazmak KESİNLİKLE yasaktır; DNS kullanılmalıdır.

- CoreDNS auto-provisions and maintains internal DNS records for all services.
- The internal FQDN syntax is `<service>.<namespace>.svc.cluster.local`.
- Never hardcode IP addresses in microservice configs; always utilize DNS names.

23

BÖLÜM: AĞ, KEŞIF & DIŞ DÜNYAYA AÇILMA | MODULE: NETWORKING, DISCOVERY & INGRESS ROUTING

Network Policies: K8s İçi Güvenlik Duvarı (Firewall)

Network Policies: Kubernetes Internal Micro-Firewalls

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Varsayılan olarak bir K8s kümesindeki her pod, diğer tüm pod'larla özgürce konuşabilir! Bu durum, oteldeki her müşterinin elini kolunu sallayarak mutfağa ve para kasasına girmesine benzer (Büyük tehlike!). NetworkPolicy, kapılara kilit takmaktır: 'Sadece beyaz şapkalı aşçılar (Backend) kasaya (Veritabanı) girebilir; müşteriler (Frontend) doğrudan giremez!'

💡 ELI5 Metaphor

🇬🇧 ENGLISH

By default, Kubernetes has a flat network: ANY pod can talk to ANY other pod! That's like letting hotel guests wander into the bank vault and restaurant freezer unhindered. A NetworkPolicy installs biometric security checkpoints: 'Only authenticated backend chefs can talk to the database; front-end visitors are completely blocked!'

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

NetworkPolicy, pod'ların ingress (gelen) ve egress (giden) ağ trafiğini etiketlere (labels), IP bloklarına (CIDR) veya namespace'lere göre filtreleyen kurallardır. DİKKAT: NetworkPolicy'nin çalışabilmesi için kümenizde Calico, Cilium veya WeaveNet gibi bir CNI (Container Network Interface) eklentisi kurulu olmalıdır. Varsayılan Flannel NetworkPolicy desteklemez. Kural yazıldığı anda hedef pod için 'varsayılan engelleme' (default-deny) aktif olur.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

NetworkPolicies enforce micro-segmentation at Layer 3/4 via Ingress/Egress packet filtering based on podSelector and namespaceSelector. CRITICAL: NetworkPolicies require an enforcing CNI plugin like Calico or Cilium. Without an enforcing CNI, policies are silently ignored! Declaring a policy toggles the selected pods into default-deny isolation.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# postgres-network-policy.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-only-backend-to-db
  namespace: coffee-shop
spec:
  podSelector:
    matchLabels:
      app: postgres-db # Bu kural veritabanı pod'larını korur
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: coffee-app # SADECE coffee-app pod'ları gelebilir!
      ports:
        - protocol: TCP
          port: 5432
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Bir pod'a NetworkPolicy bağladığınız anda, açıkça izin vermediğiniz TÜM diğer gelen bağlantılar (diğer servisler, sağlık probları vb.) anında kesilir! Probeları veya DNS trafiğini (port 53) yanlışlıkla engellememeye dikkat edin.

🇬🇧 WATCH OUT

Applying a NetworkPolicy immediately locks down all unmentioned traffic! Forgetting to allow DNS egress (port 53 UDP/TCP) will break all network lookups.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Zero-Trust güvenliği için pod'lar arası erişim kısıtlanmalıdır.
- Calico veya Cilium gibi destekleyen bir CNI eklentisi zorunludur.
- Veritabanlarına sadece yetkili backend pod'larının erişmesi garanti edilir.

- Enforce Zero-Trust architecture by restricting lateral pod-to-pod movement.
- An enforcing CNI plugin (e.g. Calico, Cilium) is mandatory for enforcement.
- Guarantees databases only accept connections from verified backend services.

24

BÖLÜM: AĞ, KEŞIF & DIŞ DÜNYAYA AÇILMA | MODULE: NETWORKING, DISCOVERY & INGRESS ROUTING

ConfigMap ve Secret: Konfigürasyonu Koddan Ayırma

Externalizing Config: ConfigMaps and Secrets with Spring Boot

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Telefon numaranız veya şifreniz değiştiğinde yeni bir telefon satın almazsınız! Aynı şekilde veritabanı şifresi değiştiğinde Spring Boot kodunu yeniden derleyip Docker imajı almamalısınız. ConfigMap: Herkesin görebileceği açık duyuru panosudur (API URL'leri, tema rengi). Secret: Kilitli çelik kasadır (Şifreler, kredi kartı API anahtarları)!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

When you change your Netflix password, you don't throw your TV in the trash and buy a new one! Similarly, when a DB password rotates, you don't recompile your Java code into a new Docker image. ConfigMaps are public bulletin boards (log levels, URLs). Secrets are locked steel safes (passwords, JWT signing keys)!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Twelve-Factor App metodolojisinin 3. kuralı: 'Konfigürasyonu koddan ayır!' K8s iki nesne sunar: 1) ConfigMap: Düz metin anahtar-değer çiftleri, 2) Secret: Base64 kodlanmış hassas veriler. Bu veriler pod'a iki şekilde aktarılır: Ortam değişkeni (Environment Variable) olarak veya dosya olarak (Volume Mount). Spring Boot, `application.yml` içindeki `${DB_PASSWORD}` değişkenlerini bu sayede sıfır kod değişikliğiyle otomatik enjekte eder.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Twelve-Factor App principle III: Strict separation of config from code. 1) ConfigMaps store non-confidential key-value pairs. 2) Secrets store sensitive credentials base64-encoded (or encrypted at rest via KMS). Injected into pods either as environment variables (`envFrom`) or mounted as files. Spring Boot seamlessly binds them into `@Value` and `@ConfigurationProperties`.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# ConfigMap ve Secret'ı Pod'a Aktarma:
spec:
  containers:
    - name: coffee-app
      image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
      env:
        - name: APP_MESSAGE # ConfigMap'ten tek değişken oku
          valueFrom:
            configMapKeyRef:
              name: spring-boot-config
              key: APP_MESSAGE
        - name: DB_PASSWORD # Secret'tan şifre oku
          valueFrom:
            secretKeyRef:
              name: spring-boot-secret
              key: DB_PASSWORD
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

K8s Secret'ları varsayılan olarak şifrelenmiş (encrypted) DEĞİLDİR; sadece Base64 ile encode edilmiştir! Yani `echo 'c2VjcmV0' | base64 -d` yazan herkes şifreyi görebilir. Gerçek güvenlik için etcd 'Encryption at Rest' veya HashiCorp Vault / AWS Secrets Manager kullanılmalıdır.

🇬🇧 WATCH OUT

Kubernetes Secrets are NOT encrypted by default; they are merely Base64-encoded! Anyone with read access can decode them. Enable KMS encryption-at-rest in etcd.

🎯 Özet Çıkarımlar

- Aynı Docker imajı dev, test ve prod ortamlarında sıfır değişiklikle çalışır.
- Ortam değişkenleri veya dosya bağlama (volume mount) ile pod'a aktarılır.
- Secret'lar hassas verileri Git reposuna koyma felaketini önler.

🎯 Key Takeaways

- The same immutable container image runs across dev, stage, and prod via config injection.
- Configs can be mounted dynamically as filesystem directories or environment vars.
- Keeps passwords and private keys out of public Git version control repositories.